

Desenvolvimento para Dispositivos Móveis

Introdução ao Flutter

Universidade Federal do Rio Grande do Norte

Introdução

- O Flutter é um framework para desenvolvimento de **interfaces multiplataforma** projetada para permitir a **reutilização de código em sistemas operacionais como iOS e Android**.
- Durante o desenvolvimento, os aplicativos Flutter são **executados em uma Máquina Virtual (VM)** que oferece a recarga imediata e mantém o estado das alterações sem a necessidade de uma recompilação completa.

```
flutter run
```

- Para a versão final, os aplicativos Flutter são compilados diretamente para código de máquina, seja para instruções Intel x64 ou ARM, ou para JavaScript, se o alvo for a web.

```
flutter build appbundle
```

Introdução

- Para executar a aplicação em um dispositivo Android:
 - Ativar as opções de Desenvolvedor e a depuração USB no seu dispositivo.
 - Conecte seu telefone ao computador com um cabo USB.
 - Usar o comando para ver se o dispositivo foi listado.

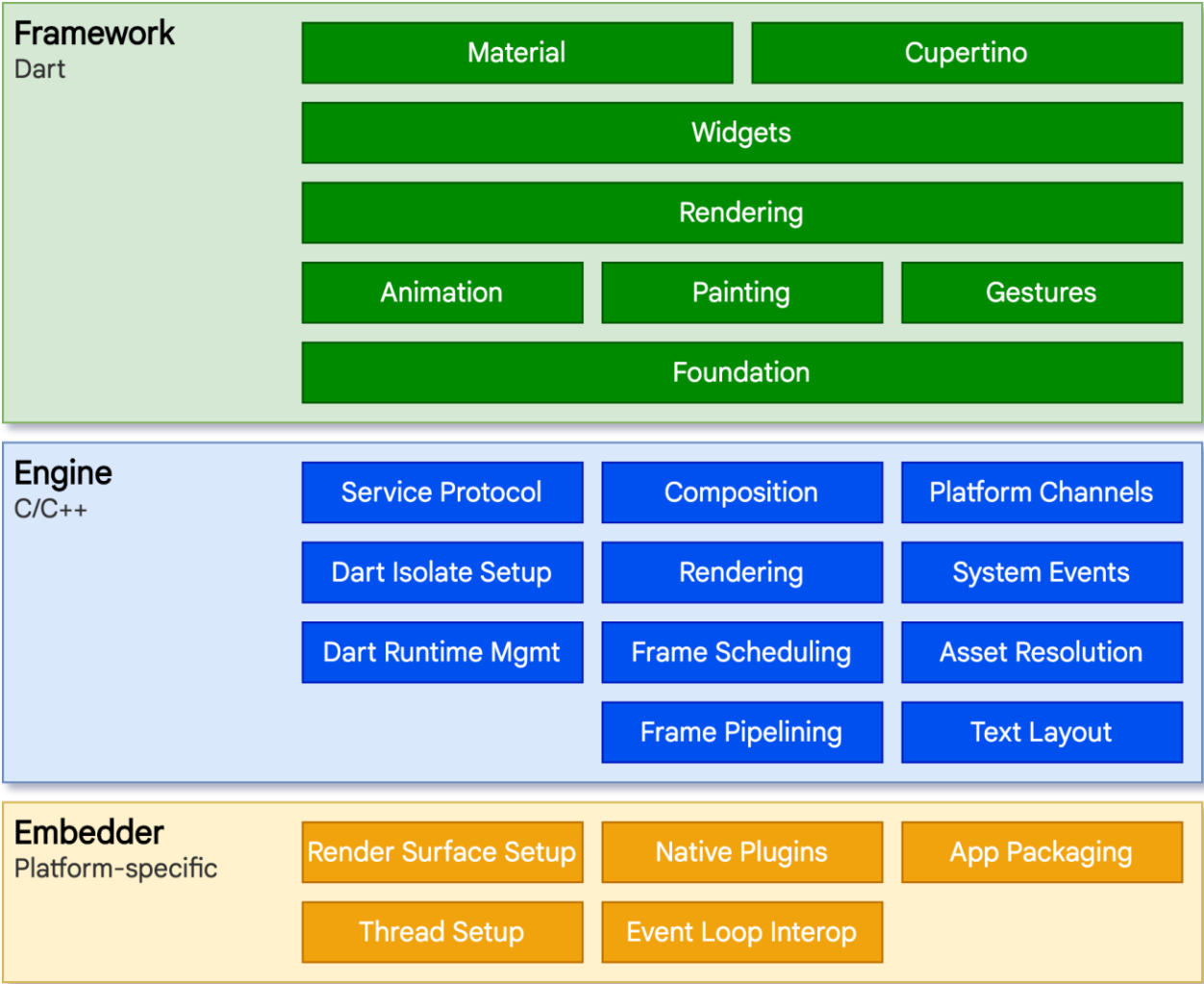
```
flutter devices
```

- Usar o comando flutter run.

Introdução

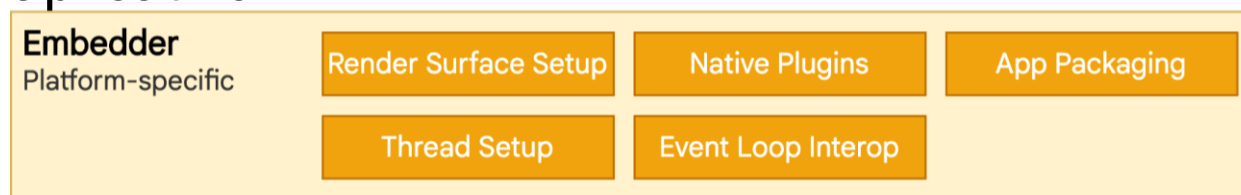
- O Flutter foi projetado como um **sistema extensível e em camadas**.
- O Flutter consiste em uma série de bibliotecas independentes, sendo que cada uma **depende da camada subjacente**.
- Nenhuma camada tem acesso privilegiado à camada abaixo e cada parte do nível do framework é projetada para ser opcional e substituível.

Arquitetura



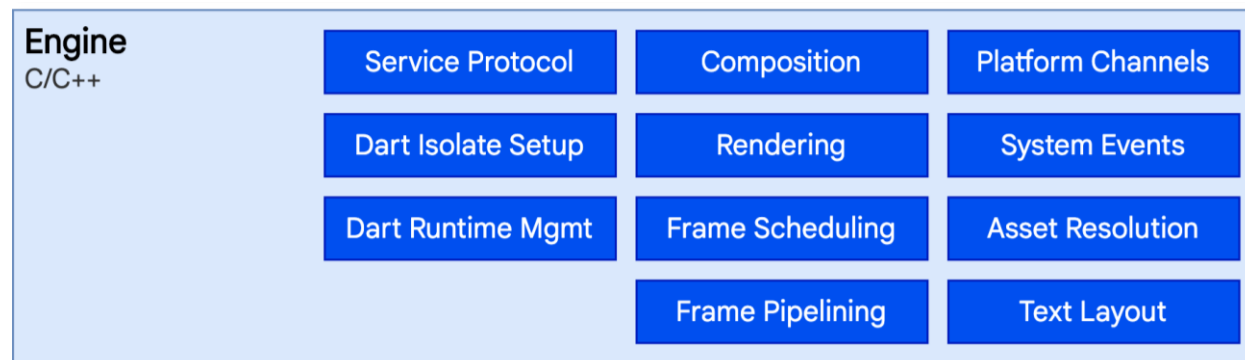
Incorporador (Embedder)

- Para o sistema operacional subjacente, os aplicativos Flutter são empacotados da mesma maneira que **qualquer outro aplicativo nativo**.
- Um incorporador (embedder) **específico da plataforma** fornece um ponto de entrada, coordena-se com o sistema operacional subjacente para acessar serviços como renderização, acessibilidade e entrada, e gerencia o loop de eventos de mensagens.
- O incorporador é escrito em uma linguagem **apropriada para a plataforma**
 - Java e C++ para Android
 - Objective-C/Objective-C++ para iOS e macOS
 - C++ para Windows e Linux.
- Usando o incorporador, o **código Flutter pode ser integrado em um aplicativo existente como um módulo**, ou o código pode constituir todo o conteúdo do aplicativo.



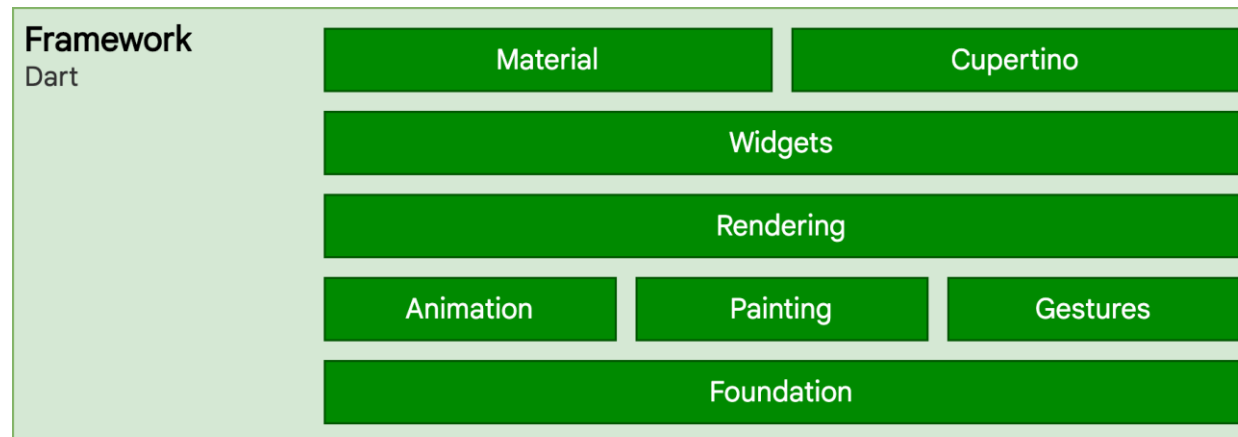
Motor Flutter

- O motor Flutter é principalmente escrito em C++ e **oferece suporte às primitivas necessárias para todos os aplicativos Flutter**.
- O motor é responsável por rasterizar cenas compostas sempre que uma nova imagem precisa ser desenhada.
- O motor fornece a implementação de baixo nível da API central do Flutter, incluindo gráficos, layout de texto, E/S de arquivo e rede, suporte à acessibilidade, arquitetura de plugins e uma cadeia de ferramentas de compilação e tempo de execução Dart.
- O motor é exposto ao framework Flutter através do **dart:ui**. Essa **biblioteca expõe as primitivas de nível mais baixo**, como classes para gerenciar a entrada, gráficos e subsistemas de renderização de texto.



Framework Flutter

- Normalmente, os desenvolvedores interagem com o Flutter por meio do **framework Flutter**, que oferece um framework moderno e reativo escrito na **linguagem Dart**.
- O framework Flutter inclui um conjunto abrangente de bibliotecas de plataforma e layout, compostas por uma série de camadas.



Framework Flutter

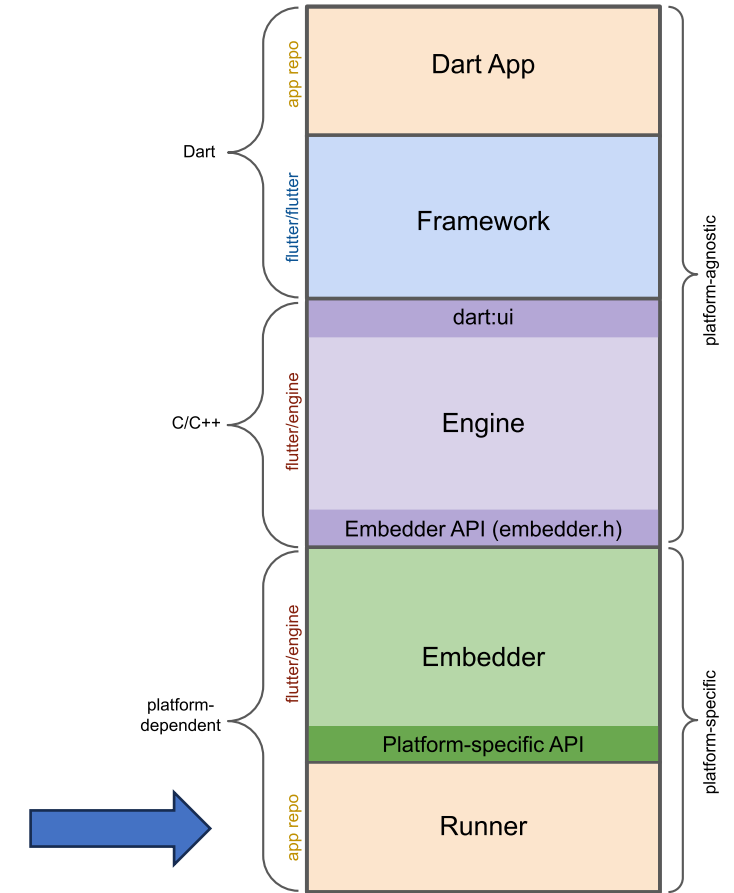
- As classes de **Foundation** proveem serviços de blocos de construção, como **animation**, **painting** e **gesture**, oferecem abstrações comumente utilizadas.
- A camada **rendering** fornece uma abstração para lidar com o layout. Com esta camada, pode-se **construir uma árvore de objetos renderizáveis**. Além disso, é possível manipular esses objetos dinamicamente, com a árvore atualizando **automaticamente o layout para refletir suas alterações**.

Framework Flutter

- A camada de **widgets** é uma abstração de composição. Cada objeto de renderização na camada de renderização tem uma classe correspondente na camada de widgets. Além disso, a camada de widgets permite definir combinações de classes que podem ser reutilizadas. **Esta é a camada na qual o modelo de programação reativa é introduzido.**
- As bibliotecas **Material** e **Cupertino** oferecem conjuntos abrangentes de controles que utilizam as primitivas de composição da camada de widgets para implementar design Material ou iOS.

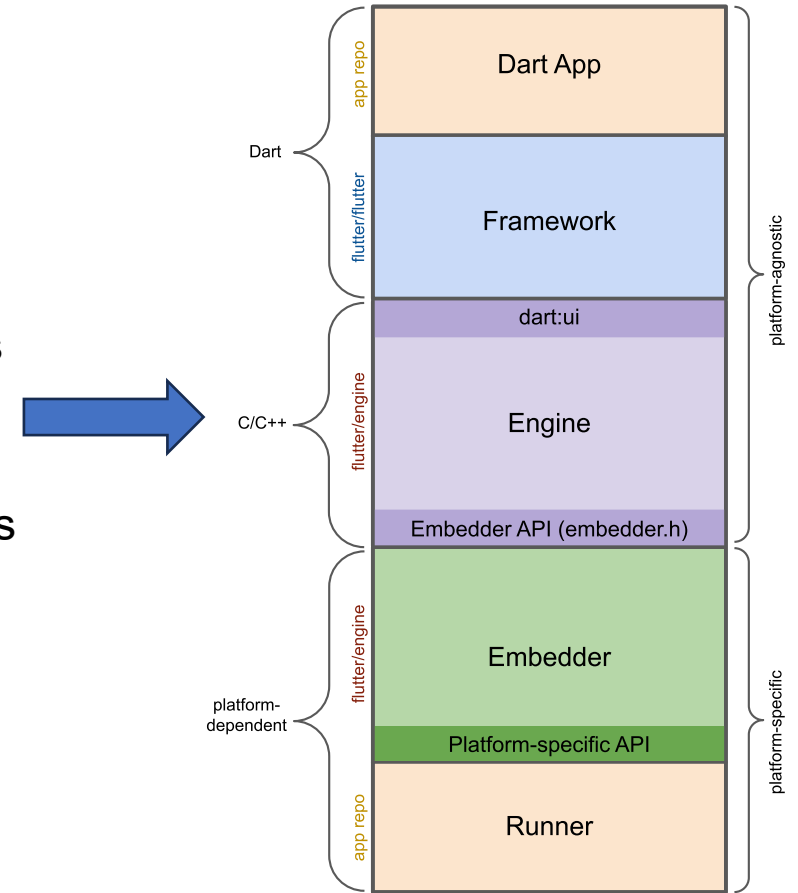
Anatomia de um App

- Runner
 - Compõe as partes expostas pela API específica da plataforma do Embedder em um pacote de aplicativo executável na plataforma de destino.
- Embedder
 - Coordena com o sistema operacional subjacente para acessar serviços como superfícies de renderização, acessibilidade e entrada.
 - Gerencia o loop de eventos.
 - Expõe uma API específica da plataforma para integrar o Embedder em aplicativos



Anatomia de um App

- Engine
 - Responsável por rasterizar cenas compostas.
 - Fornece a implementação de baixo nível das APIs principais do Flutter
 - Expõe sua funcionalidade para o framework usando a API `dart:ui`.
 - Integra-se com uma plataforma específica usando a Engine's Embedder API.
- Framework
 - Fornecer uma API de nível mais alto para construir aplicativos de alta qualidade
 - Compor a árvore de widgets do aplicativo em uma cena.
- Dart app
 - Compor widgets na interface do usuário desejada.
 - Implementa a lógica de negócios.
 - Propriedade do desenvolvedor do aplicativo.



Interface de Usuário Reativa

- Flutter é um framework de IU **reativo e declarativo**, no qual o desenvolvedor fornece um mapeamento do estado da aplicação para o estado da interface.
- Framework assume a tarefa de **atualizar a interface em tempo de execução** quando o estado da aplicação muda.
 - Modelo inspirado no React do facebook.

Interface de Usuário Reativa

- O Flutter desacopla a interface do usuário de seu estado subjacente.
 - O desenvolvedor apenas cria a descrição da IU, e o framework se encarrega de usar essa única configuração para criar e/ou atualizar a interface.
- No Flutter, **widgets são representados por classes imutáveis que são usadas para configurar uma árvore de objetos.**
- Esses widgets são usados para gerenciar uma árvore separada de objetos para o layout, que é então usada para gerenciar uma árvore separada de objetos para composição.
- O Flutter é, em sua essência, uma série de mecanismos para percorrer **eficientemente as partes modificadas das árvores**, converter árvores de objetos em árvores de objetos de nível inferior e propagar alterações por essas árvores.

Widgets

- Widgets são os blocos de construção fundamentais em Flutter.
- Eles descrevem **como a interface deve ser exibida** com base em sua configuração e estado atual.
- Um **widget** declara sua interface de usuário sobrescrevendo o método `build()`, que é uma função que converte o estado em UI.
- O método **`build()`** é, por design, rápido para ser executado e deve ser livre de efeitos colaterais, permitindo que o framework o chame sempre que necessário (potencialmente, tão frequentemente quanto uma vez por quadro renderizado).
 - Por isso, recomenda-se o uso de construtores constantes.

Widgets

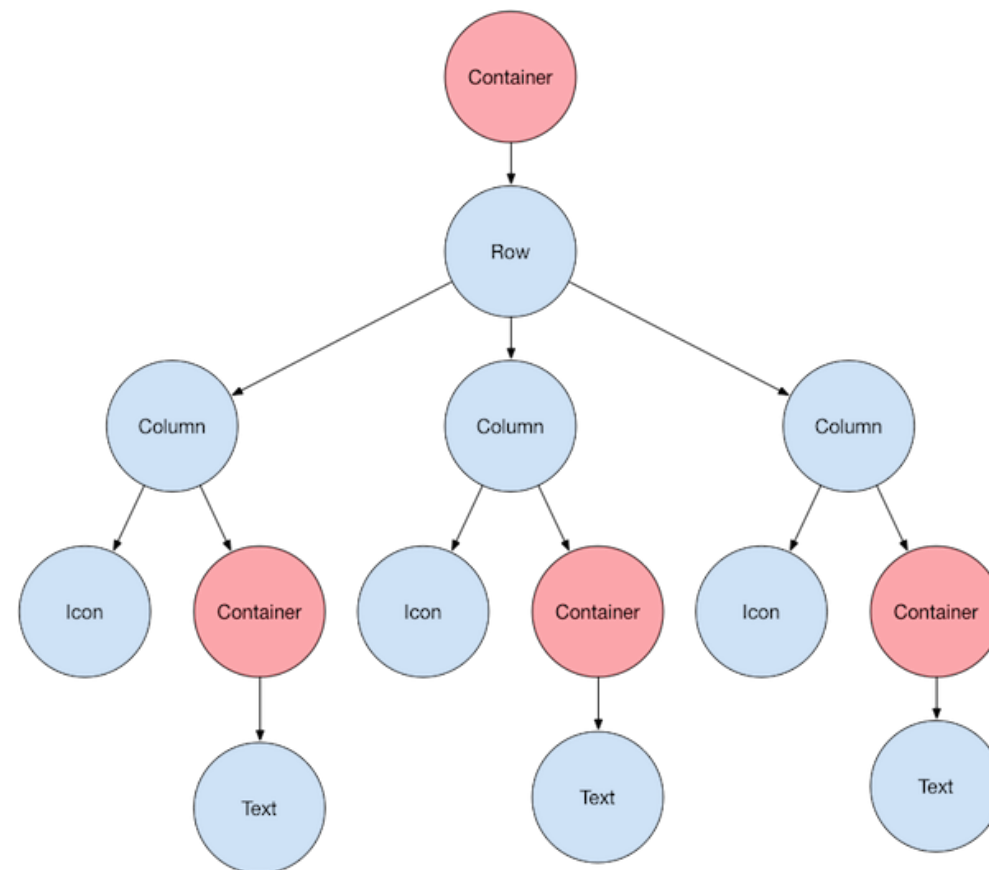
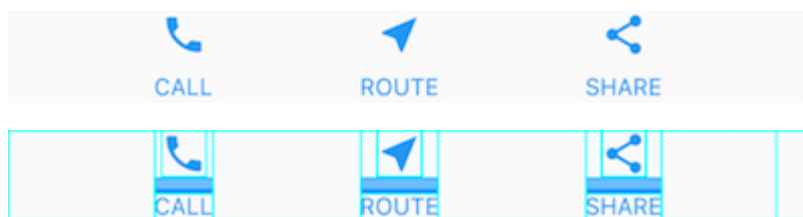
- A ideia central é que o desenvolvedor possa **construir a interface do usuário a partir de widgets**.
- Quando o estado de um widget muda, ele é reconstruído.
- O Flutter então compara a versão atual com a versão anterior.
- Isso permite determinar as **mudanças mínimas necessárias** na árvore de renderização subjacente para transitar de um estado para o próximo.
 - Reduz a necessidade de recriar toda a interface do usuário, economizando recursos.

Widgets

- O núcleo do mecanismo de layout do Flutter são os widgets.
- No Flutter, **praticamente tudo é um widget**, inclusive modelos de layout.
 - imagens, ícones e textos são todos widgets
- Elementos não visuais também são widgets
 - linhas, colunas e grades que organizam, restringem e alinham os widgets visíveis.

Composição

- Cria-se um layout compondo widgets para construir widgets mais complexos.



FONTE: <https://docs.flutter.dev/ui/layout>

Hello World

```
import 'package:flutter/material.dart';

void main() {
  runApp(const Text("Hello World",
    textDirection: TextDirection.ltr));
}
```



Hello World

- A função `runApp()` **recebe o Widget** fornecido e o torna a **raiz da árvore de widgets**.
- Neste exemplo, a árvore de widgets consiste apenas no widget `Text`.
- O framework força o widget raiz a cobrir a tela.
- Como não foi utilizado nenhum `Widget` para organizar os elementos na tela, temos o texto na parte superior da tela.
 - A direção do texto precisa ser especificada neste caso; quando o widget `MaterialApp` é usado, isso é feito automaticamente.

Hello World

```
import 'package:flutter/material.dart';

void main() {
  runApp(const ColoredBox(
    color: Colors.red,
    child: Text(
      "Hello World",
      textDirection: TextDirection.ltr
    ),
  ));
}
```



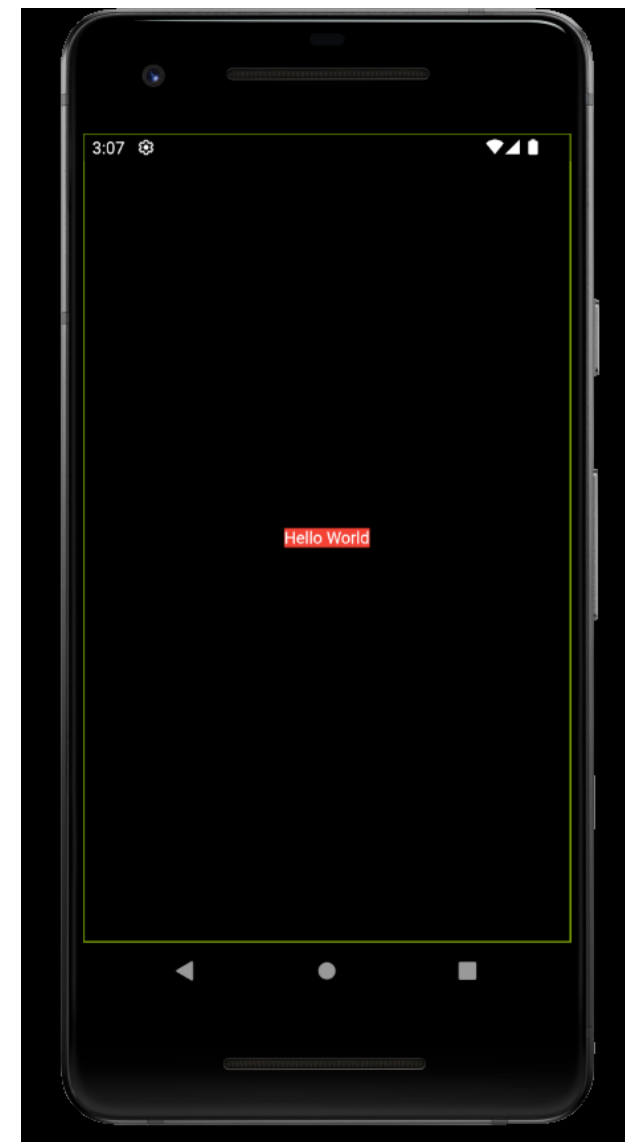
Hello World

- Coloredbox é um widget que pinta a área utilizada pelo widget filho.
- Para que o texto fique no centro é possível utilizar o Widget Center.

Hello World

```
import 'package:flutter/material.dart';

void main() {
  runApp(const Center(
    child: ColoredBox(
      color: Colors.red,
      child: Text(
        "Hello World",
        textDirection: TextDirection.ltr
      ),
    ),
  ));
}
```



Widgets básicos

- **Text:** O widget Text permite criar um texto estilizado.
- **Row, Column:** Esses widgets flexíveis permitem criar layouts flexíveis nas direções horizontal (Row) e vertical (Column). O design desses objetos é baseado no modelo de layout flexbox da web.
- **Stack:** Em vez de ser orientado linearmente (seja horizontalmente ou verticalmente), um widget Stack permite colocar widgets uns sobre os outros.
- **Container:** Permite criar um elemento visual retangular. Um container pode ser decorado com um BoxDecoration, como um plano de fundo, uma borda ou uma sombra. Um Container também pode ter margens, preenchimentos e restrições aplicadas ao seu tamanho.

Material Design

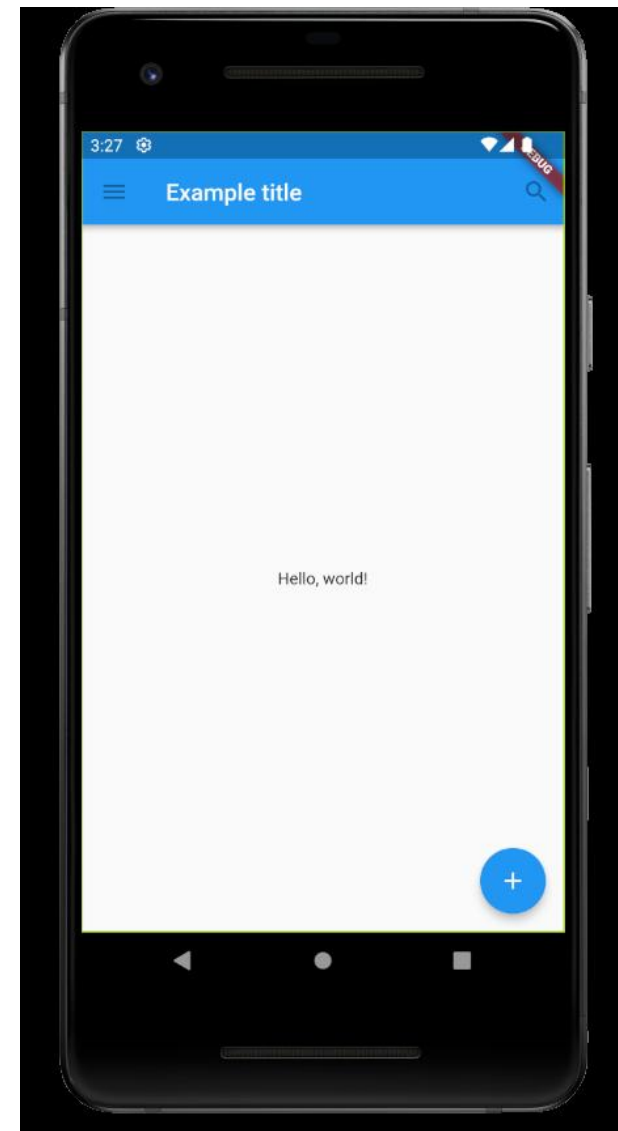
- O Flutter fornece uma série de widgets que ajudam a criar aplicativos que seguem o Material Design.
- Um aplicativo para Material Design começa com o widget MaterialApp, que constrói vários widgets úteis na raiz do aplicativo
 - Exemplo: Navigator, que gerencia uma pilha de widgets identificados por strings, também conhecidos como “rotas”.
- Usar o widget MaterialApp é completamente opcional, mas é uma boa prática, pois fornece uma estrutura sólida para o desenvolvimento de aplicativos seguindo as diretrizes do Material Design.

Material Design

```
void main() {  
  runApp(  
    const MaterialApp(  
      title: 'Flutter Tutorial',  
      home: TutorialHome(),  
    ),  
  );  
}
```



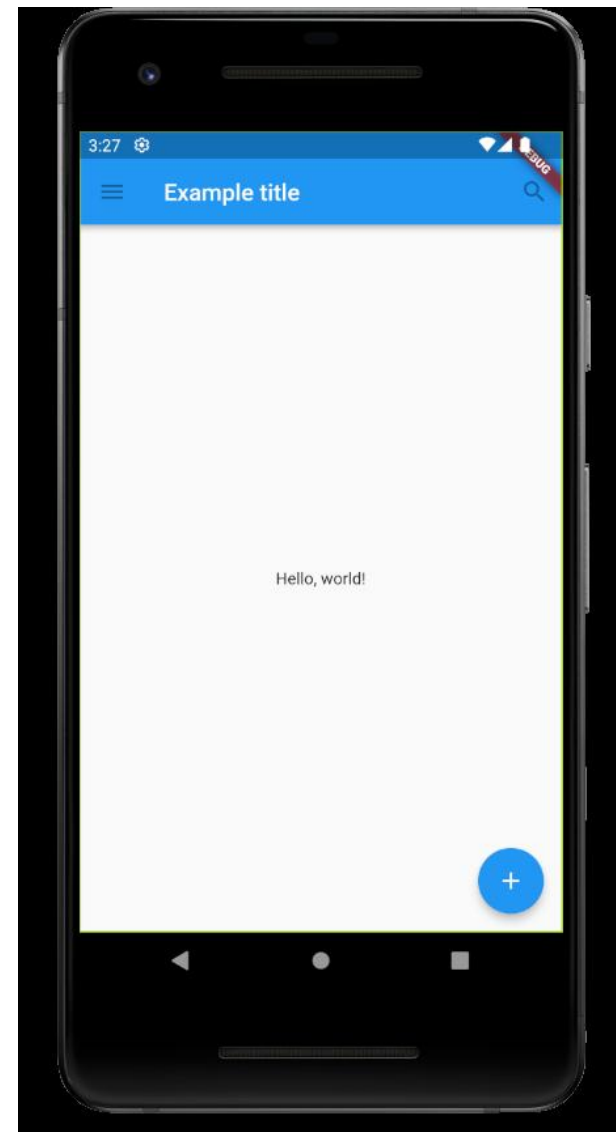
```
class TutorialHome extends StatelessWidget {  
  const TutorialHome({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        leading: const IconButton(  
          icon: Icon(Icons.menu),  
          tooltip: 'Navigation menu',  
          onPressed: null,  
        ),  
        title: const Text('Example title'),  
        actions: const [  
          IconButton(  
            icon: Icon(Icons.search),  
            tooltip: 'Search',  
            onPressed: null,  
          ),  
        ],  
      ),  
      body: const Center(  
        child: Text('Hello, world!'),  
      ),  
      floatingActionButton: const FloatingActionButton(  
        tooltip: 'Add', // used by assistive technologies  
        onPressed: null,  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```



Material Design

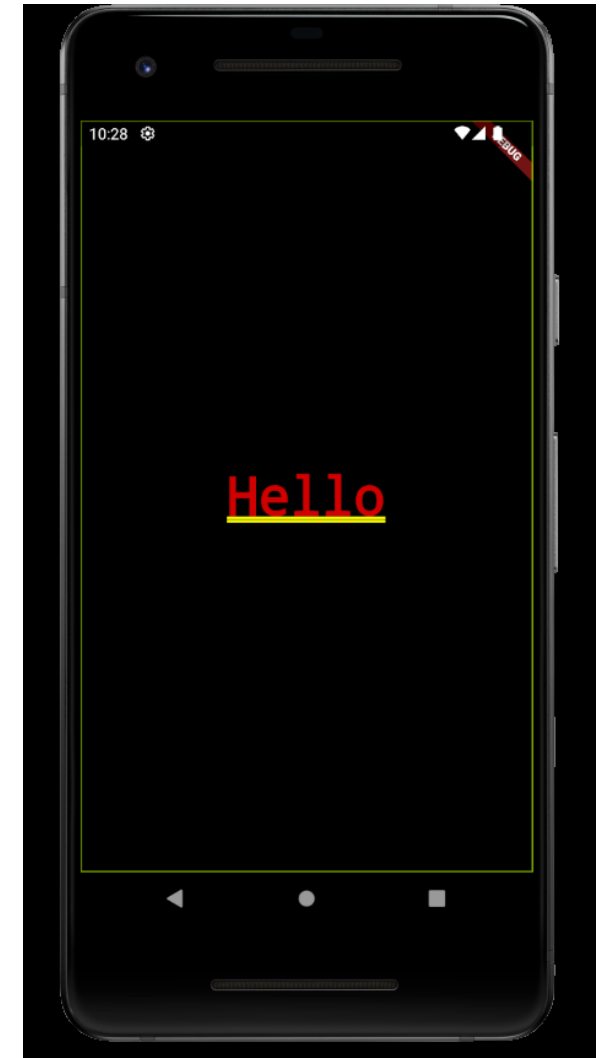
```
void main() {  
  runApp(  
    const MaterialApp(  
      title: 'Flutter Tutorial',  
      home: TutorialHome(),  
    ),  
  );  
}
```

```
class TutorialHome extends StatelessWidget {  
  const TutorialHome({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        leading: const IconButton(  
          icon: Icon(Icons.menu),  
          tooltip: 'Navigation menu',  
          onPressed: null,  
        ),  
        title: const Text('Example title'),  
        actions: const [  
          IconButton(  
            icon: Icon(Icons.search),  
            tooltip: 'Search',  
            onPressed: null,  
          ),  
        ],  
      ),  
      body: const Center(  
        child: Text('Hello, world!'),  
      ),  
      floatingActionButton: const FloatingActionButton(  
        tooltip: 'Add', // used by assistive technologies  
        onPressed: null,  
        child: Icon(Icons.add),  
      ),  
    ),  
  );  
}
```



MaterialApp

- O **MaterialApp** é um widget de conveniência que envolve vários widgets comumente necessários para aplicativos de **Material Design**.
- O **MaterialApp** configura seu `WidgetsApp.textStyle` com um estilo de texto vermelho/amarelo feio que tem a intenção de alertar o desenvolvedor de que seu aplicativo não definiu um estilo de texto padrão.
- Normalmente, o Scaffold do aplicativo constrói um widget Material cujo `Material.textStyle` padrão define o estilo de texto para todo o scaffold.



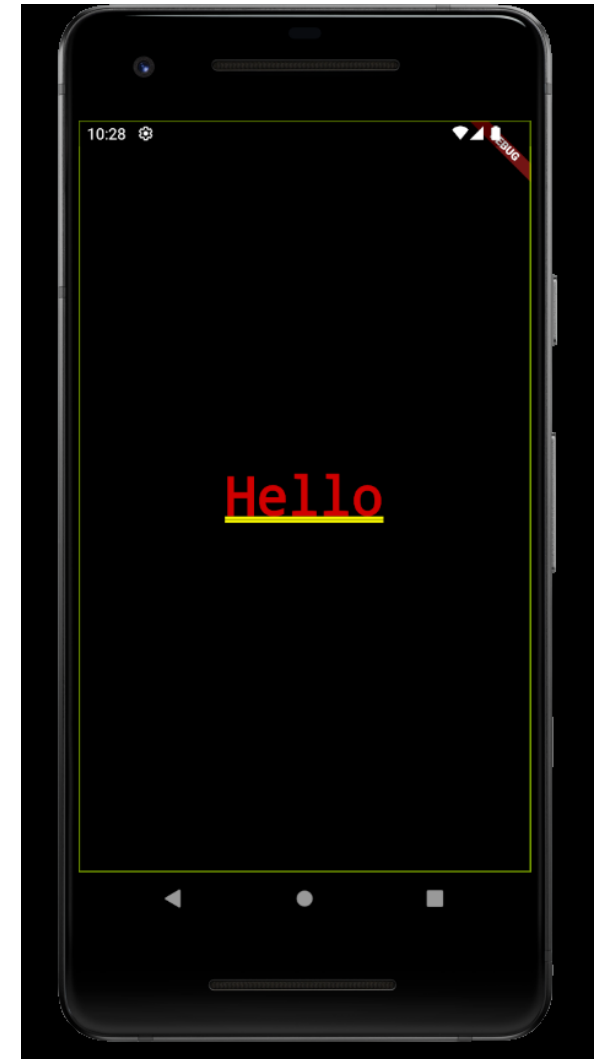
MaterialApp

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

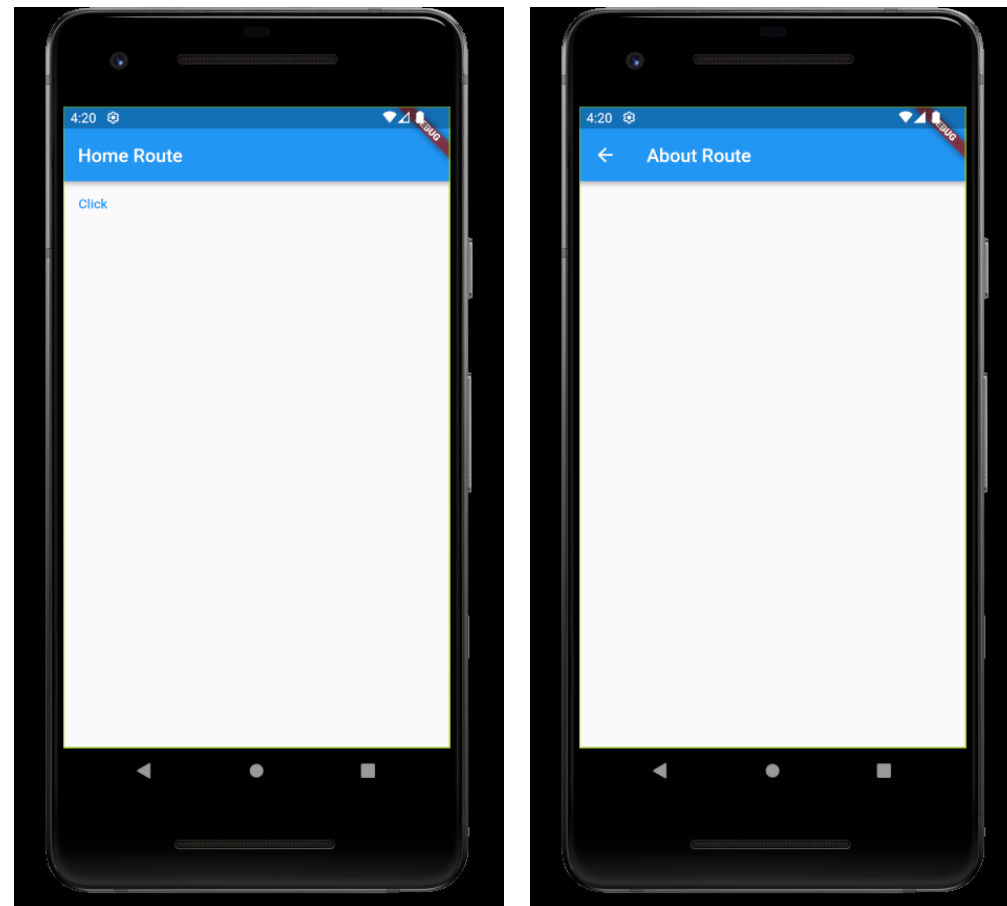
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: const Center(child: Text("Hello")),
    );
  }
}
```



MaterialApp

```
void main() {  
  runApp(MaterialApp(  
    routes: <String, WidgetBuilder>{  
      '/': (BuildContext context) {  
        return Scaffold(  
          appBar: AppBar(  
            title: const Text('Home Route'),  
          ),  
          body: TextButton(  
            child: const Text("Click"),  
            onPressed: () => Navigator.pushNamed(context, "/about")  
          ),  
        );  
      },  
      '/about': (BuildContext context) {  
        return Scaffold(  
          appBar: AppBar(  
            title: const Text('About Route'),  
          ),  
        );  
      },  
    },  
  ));  
}
```



MaterialApp

- No Widget MaterialApp são encontradas diversas propriedades que estão relacionadas a configuração de um aplicativo que utiliza o Material Design.
- Para tratar do design da tela será utilizado o Widget Scaffold

```
const MaterialApp({  
  this.home,  
  Map<String, WidgetBuilder> this.routes = const <String, WidgetBuilder>{},  
  this.initialRoute,  
  this.title = '',  
  this.color,  
  this.theme,  
  this.darkTheme,  
  this.locale,  
  this.localizationsDelegates,  
  this.supportedLocales = const <Locale>[Locale('en', 'US')]  
  (...)  
})
```

Scaffold

- Scaffold implementa a estrutura visual básica do Material Design.
- Esta classe fornece APIs para elementos característicos como gavetas (drawers), folhas inferiores (bottom sheets), ActionBar, Botão de Ação Flutuante, etc.

```
class Scaffold extends StatefulWidget {  
  const Scaffold({  
    this.appBar,  
    this.body,  
    this.floatingActionButton,  
    this.drawer,  
    this.bottomNavigationBar,  
    this.bottomSheet,  
    this.backgroundColor  
    (...)  
  });  
}
```


Widget State

- O Flutter apresenta duas classes principais de widgets: widgets stateful e widgets stateless.
- **Muitos widgets não possuem estado mutável:** eles não têm propriedades que mudam ao longo do tempo (por exemplo, um ícone ou um rótulo). **Esses widgets são subclasses de StatelessWidget.**

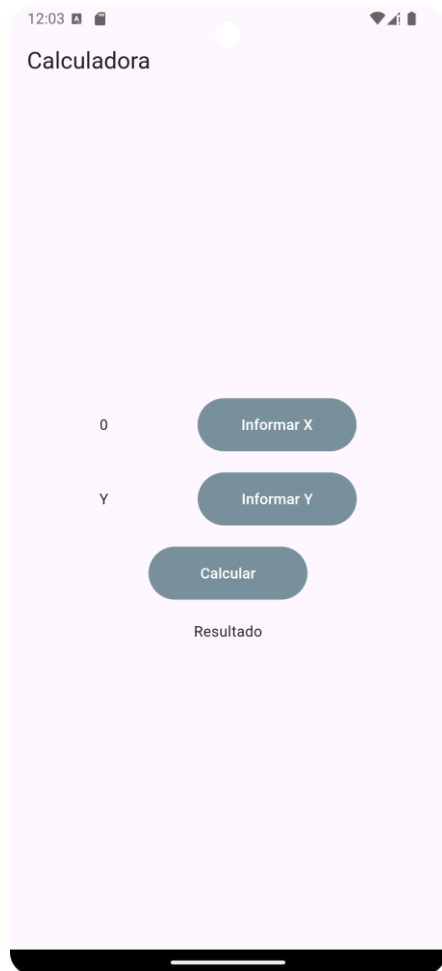
Widget State

- Se as **características** de um widget **precisarem mudar** com base na interação do usuário ou outros fatores, esse widget será **stateful**.
 - Por exemplo, se um widget tiver um contador que aumenta sempre que o usuário tocar em um botão, então o valor do contador é o estado desse widget.
- Quando esse valor muda, o widget precisa ser **reconstruído** para atualizar sua parte da interface do usuário.
- Esses widgets são subclasses de **StatefulWidget** e (porque o próprio widget é imutável) eles armazenam o estado mutável em uma classe separada que é uma subclasse de **State**.
- Os **StatefulWidgets** não possuem um método `build()`; em vez disso, sua interface do usuário é construída por meio de seu objeto **State**.

Widget State

- Sempre que um objeto **State** mudar, deve-se chamar **setState()** para sinalizar ao framework que atualize a interface do usuário chamando novamente o método build do State.
- **Ter objetos de estado separados dos widgets permite que outros widgets tratem tanto widgets stateless quanto widgets stateful da mesma maneira**, sem se preocupar em perder o estado.
- Em vez de precisar manter um filho para preservar seu estado, o pai pode criar uma nova instância do filho a qualquer momento sem perder o estado persistente do filho. O framework faz todo o trabalho de encontrar e reutilizar objetos de estado existentes quando apropriado.

Atividade – Calculadora (parte 1)



- Crie um aplicativo com duas telas.
- Navegue da tela 1 para tela 2 .

Referências

- <https://dart.dev/>
- <https://dart-tutorial.com/>
- <https://docs.flutter.dev/ui>
- <https://docs.flutter.dev/ui/widgets>
- <https://docs.flutter.dev/ui/layout>
- <https://docs.flutter.dev/resources/architectural-overview>